

数据库完整性

0.概述

1. 数据库完整性的定义:

- 数据的正确性: **是指数据是符合现实世界语义, 反映了当前实际状况的**
- 数据的相容性: **是指数据库同一对象在不同关系表中的数据是符合逻辑的**

2. 完整性和安全性的比较:

- 完整性: 防止数据库中存在不正确的数据; 对象: **不合语义的、不正确的数据**
- 安全性: 保护数据库防止恶意的破坏和非法的存取; 对象: **非法用户和非法操作**

3. 完整性机制:

- **提供定义完整性约束条件的机制**: 也叫完整性规则, SQL标准中包括关系模型中的实体完整性、参照完整性和用户定义完整性
- **提供完整性检查的方法**: 检查是否满足完整性约束条件; 一般在INSERT、UPDATE、DELETE语句执行后, 也可以在事务提交时检查
- **违约处理**: 当DBMS发现用户的操作违背了完整性约束条件就采取动作: 拒绝执行该操作; 级联执行其他操作

4. 完整性约束定义的分类:

- 按定义对象:
 - 与表有关的约束
 - 域约束
 - 断言
- 按约束状态:
 - 静态约束: 要求数据库在任何时候都要满足的约束
 - 动态约束: 在数据库改变状态时要满足的约束
- SQL支持如下约束:
 - 静态约束: 与表有关的约束, 域约束; 断言 (careful!)
 - 动态约束: 触发器

触发器不仅可以检查数据完整性,还可以进行数据处理\实现某些业务处理逻辑

1.实体完整性

1.1 定义

在 `CREATE TABLE` 命令用 `PRIMARY KEY` 定义主码, 或者用 `UNIQUE + NOT NULL` 定义唯一码

单列构成的码可以定义为列级约束条件或表级约束条件;多列构成的码只能用表级约束

示例:

❑ [例5.1] 将Student表中的Sno列定义为码

❑ 在列级定义主码

```
CREATE TABLE Student (  
    Sno CHAR(9) PRIMARY KEY,  
    Sname CHAR(20) NOT NULL,  
    Ssex CHAR(2),  
    Sage SMALLINT,  
    Sdept CHAR(20)  
);
```

❑ 在表级定义主码

```
CREATE TABLE Student (  
    Sno CHAR(9),  
    Sname CHAR(20) NOT NULL,  
    Ssex CHAR(2),  
    Sage SMALLINT,  
    Sdept CHAR(20),  
    PRIMARY KEY (Sno)  
);
```

❑ [例5.2] 将SC表中的Sno, Cno列组定义为码

```
CREATE TABLE SC (  
    Sno CHAR(9) NOT NULL,  
    Cno CHAR(4) NOT NULL,  
    Grade SMALLINT,  
    PRIMARY KEY (Sno,Cno)    /*只能在表级定义主码*/  
);
```

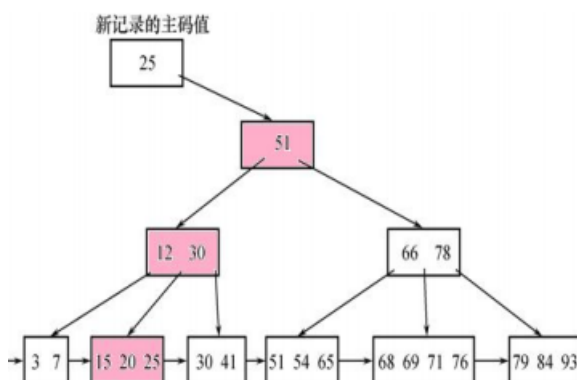
1.2 实体完整性检查和违约处理

插入或对主码列进行更新操作时，关系数据库管理系统按照实体完整性规则自动进行检查。包括：

- 检查主码的各个列是否为空，只要有一个为空就拒绝插入或修改
- 检查主码值是否唯一，如果不唯一则拒绝插入或修改

检查的一种方法是进行全表扫描,但是非常耗时

另一种方法是在主码上自动建立一个索引



❑ 例如：

- 新插入记录的主码值是25
- 通过主码索引，从B+树的根结点开始查找
- 读取3个结点：
 - 根结点 (51)
 - 中间结点 (12 30)
 - 叶结点 (15 20 25)
- 该主码值已经存在，不能插入这条主码值为25的新记录

2.参照完整性

2.1 定义

在 `CREATE TABLE` 命令中用 `FOREIGN KEY` 短语定义外码，用 `REFERENCES` 短语指明这些外码参照哪些表的主码

- 单列构成的外码，既可以作为列级约束、也可以作为表级约束来说明
- 多列构成的外码，只能作为表级约束来说明

示例:

□[例5.3] 定义SC中的参照完整性

```
CREATE TABLE SC (  
    Sno CHAR(9) NOT NULL REFERENCES Student(Sno),  
        /* 在列级定义参照完整性 - 外码 Sno */  
    Cno CHAR(4) NOT NULL,  
    Grade SMALLINT,  
    PRIMARY KEY (Sno, Cno),  
        /* 在表级定义实体完整性 - 主码 (Sno,Cno) */  
    FOREIGN KEY (Cno) REFERENCES Course(Cno)  
        /* 在表级定义参照完整性 - 外码 Cno */  
);
```

/* 单列构成的主码和外码，可以作为列级约束、也可以作为表级约束进行说明 */
/* 多列构成的主码和外码，只能作为表级约束进行说明 */

2.2 参照完整性检查和违约处理

示例:

□例如，对参照表SC和被参照表Student，有四种可能破坏参照完整性的情况：

- ① 在参照表SC中增加一个元组，该元组的Sno列的值v在被参照表Student中不存在（找不到一个Student元组，其Sno列的值与v相等）
- ② 修改参照表SC中的一个元组，修改后该元组的Sno列的值new_v在被参照表Student中不存在（找不到一个Student元组，其Sno列的值与new_v相等）
- ③ 从被参照表Student中删除一个元组，造成参照表SC中某些元组的Sno列的值在被参照表Student中不存在
- ④ 修改被参照表Student中一个元组的Sno列，造成参照表SC中某些元组的Sno列的值在被参照表Student中不存在

处理:

被参照表（例如Student）	参照表（例如SC）	违约处理
可能破坏参照完整性 ←	插入元组	拒绝
可能破坏参照完整性 ←	修改外码值	拒绝
删除元组 →	可能破坏参照完整性	拒绝 / 级连删除 / 设置为空值
修改主码值 →	可能破坏参照完整性	拒绝 / 级连修改 / 设置为空值

违约处理:

- **拒绝执行**: 不允许操作执行, default
- **级联操作**: 当删除或修改被参照表的一个元组造成了与参照表的不一致, 则删除或修改参照表中的所有造成不一致的元组
- **设置为空值**:

当删除或修改被参照表的一个元组时造成了不一致, 则将参照表中的所有造成不一致的元组的对应列设置为空值。

对于参照完整性, 除了应该定义外码, 还应定义**外码列是否允许空值**

示例:

□ 例如, 有下面2个关系

- 学生 (学号, 姓名, 性别, 专业号, 年龄)
- 专业 (专业号, 专业名)

外码

- 假设专业表中某个元组被删除, 专业号为12
- 按照设置为空值的策略, 就要把学生表中专业号=12的所有元组的专业号设置为空值
- 对应语义: 某个专业删除了, 该专业的所有学生专业未定, 等待重新分配专业

□[例5.4] 显式说明参照完整性的违约处理示例

```
CREATE TABLE SC (  
    Sno CHAR(9) NOT NULL,  
    Cno CHAR(4) NOT NULL,  
    Grade SMALLINT,  
    PRIMARY KEY(Sno,Cno),  
    FOREIGN KEY (Sno) REFERENCES Student(Sno)  
        ON DELETE CASCADE /*级联删除SC表中相应的元组*/  
        ON UPDATE CASCADE, /*级联更新SC表中相应的元组*/  
    FOREIGN KEY (Cno) REFERENCES Course(Cno)  
        ON DELETE NO ACTION  
        /*当删除course表中的元组造成了与SC表不一致时拒绝删除*/  
        ON UPDATE CASCADE  
        /*当更新course表中的cno时，级联更新SC表中相应的元组*/  
);
```

2.3 定义列级约束的内容

```
{ NOT NULL |  
  [ CONSTRAINT constraint_name ]  
  UNIQUE  
  | PRIMARY KEY  
  | CHECK ( search_condition )  
  | REFERENCES table_name [ ( column_name ) ]  
    [ ON DELETE CASCADE | RESTRICT | SET NULL ]  
    [ ON UPDATE CASCADE | RESTRICT | SET NULL ] }
```

3.用户定义的完整性

3.1 定义

用户定义的完整性是针对某一具体应用的数据必须满足的语义要求

关系数据库管理系统提供了定义和检验用户定义完整性的机制，不必由应用程序承担

3.2 具体操作

针对列和元组:

❑在执行 **CREATE TABLE** 命令时，给出用户定义的完整性的定义

❑列上的约束条件

➤列值非空：**NOT NULL**

➤列值唯一：**UNIQUE**

➤定义列值应该满足的条件：**CHECK <条件表达式>**

• <条件表达式>仅涉及到当前的单个列

❑元组上的约束条件

➤列值唯一：**UNIQUE(<列名> [, <列名>]...)**

➤定义列值应该满足的条件：**CHECK <条件表达式>**

• <条件表达式>可能涉及单个或多个列

4.完整性约束命名字句

4.1 完整性约束的定义与修改

- 在 **CREATE TABLE** 语句中定义完整性约束
- 定义时可以对完整性约束命名,方便使用
- 可以使用 **ALTER TABLE** 语句来修改完整性约束定义,包括新的完整性约束定义和原先的删除

4.2 完整性约束命名字句

1 | **CONSTRAINT** <完整性约束条件名><完整性约束条件>

<完整性约束条件>包括**NOT NULL**、**UNIQUE**、**PRIMARY KEY**短语、**FOREIGN KEY**短语、**CHECK**短语等

示例:

创建完整性约束 1

❑[例5.10] 建立学生登记表**Student**，要求学号在**90000~99999**之间，姓名不能取空值，年龄小于**30**，性别只能是“男”或“女”。

```
CREATE TABLE Student (  
    Sno NUMERIC(6)  
    CONSTRAINT C1 CHECK (Sno BETWEEN 90000 AND 99999),  
    Sname CHAR(20) CONSTRAINT C2 NOT NULL,  
    Sage NUMERIC(3) CONSTRAINT C3 CHECK (Sage < 30),  
    Ssex CHAR(2) CONSTRAINT C4 CHECK (Ssex IN ('男', '女')),  
    CONSTRAINT StudentKey PRIMARY KEY(Sno)  
);
```

➤在**Student**表上建立了5个约束条件，包括：主码约束（命名为**StudentKey**）以及**C1**、**C2**、**C3**、**C4**四个列级约束。

- ❑ [例5.12] 去掉例5.10 Student表中对性别的限制。

```
ALTER TABLE Student DROP CONSTRAINT C4;
```

- ❑ [例5.13] 修改表Student中的约束条件，要求学号改为在900000 ~ 999999之间，年龄由小于30改为小于40。

➤ 可以先删除原来的约束条件，再增加新的约束条件

```
ALTER TABLE Student DROP CONSTRAINT C1;
```

```
ALTER TABLE Student
```

```
ADD CONSTRAINT C1 CHECK (Sno BETWEEN 900000 AND 999999);
```

```
ALTER TABLE Student DROP CONSTRAINT C3;
```

```
ALTER TABLE Student ADD CONSTRAINT C3 CHECK(Sage < 40);
```

5.域中的完整性限制

- ❑ 在用CREATE DOMAIN语句定义域时，可以定义域上的完整性约束，即该域中的值应该满足的约束条件。

- ❑ 可以方便对所有使用相同‘域’的列的完整性约束的统一定义

- ❑ 例5.15 至 例5.17

```
CREATE DOMAIN GenderDomain CHAR(2)
```

```
CONSTRAINT GD CHECK(VALUE IN ('男', '女'));
```

```
ALTER DOMAIN GenderDomain DROP CONSTRAINT GD;
```

```
ALTER DOMAIN GenderDomain
```

```
ADD CONSTRAINT GDD CHECK(VALUE IN ('1', '0'));
```

6.断言

6.1 说明

在SQL中使用 CREATE ASSERTION 语句来指定更一般性的约束

断言可以涉及多个表的或聚集操作的比较复杂的完整性约束

任何使得断言不为真值的操作都会被拒绝执行

断言的复杂度会导致系统在检测和维护时的高开销

6.2 断言相关语句

```
1  # 1.创建断言
2  CREATE ASSERTION <断言名><CHECK 子句>
3  # 每个断言都要一个名字，<CHECK 子句>中的约束条件与WHERE子句的条件表达式类似
4
5  # 2.删除断言
6  DROP ASSERTION <断言名>
```

在断言语句中不能使用聚集函数和分组函数的语句。因为断言语句通常用于验证数据的完整性和一致性，而聚集函数用于对数据进行汇总和统计。在断言语句中使用聚集函数会导致语法错误。如果需要在断言语句中使用聚集函数，可以先将数据查询出来，然后在程序中进行相应的判断和处理。

7.触发器

7.1 定义

触发器是用户定义在关系表上的一类由事件驱动的特殊过程。

- 触发器保存在数据库服务器中
- 任何用户对表的增删改操作均由服务器自动激活相应的触发器
- 触发器具有更精细和更强大的数据控制能力

7.2 定义触发器1

语法格式：

```
1  CREATE TRIGGER <触发器名>
2  { BEFORE | AFTER } <触发事件> ON <表名>
3  REFERENCING NEW | OLD ROW AS <变量>
4  FOR EACH { ROW | STATEMENT }
5  [ WHEN <触发条件> ] <触发动作体>
```

触发器又叫“事件——条件——动作”规则。

当特定的系统事件“<触发事件> ON <表名>”发生时，对规则的<触发条件>进行检查，如果<触发条件>成立则执行规则中的动作“<触发动作体>”，否则不执行。

<触发动作体>可以很复杂，通常是一段SQL存储过程。

详细的触发器定义命令：


```

CREATE TRIGGER trigger_name { BEFORE | AFTER }
{ INSERT | DELETE
  | UPDATE [ OF colname { , colname ... } ] }
ON table_name
[ REFERENCING corr_name_def { , ..... } ]
[ FOR EACH ROW | FOR EACH STATEMENT ]
[ WHEN ( search_condition ) ]
{ statement
  | BEGIN ATOMIC statement; { statement; ... } END

```

触发事件

触发条件 / 执行方式

结果事件 / 触发动作体

7.3 定义触发器2

表的拥有者才可以在表上创建触发器。

触发器名：

- 触发器名可以包含模式名，也可以不包含模式名
- 同一模式下，触发器名必须是唯一的
- 触发器名和表名必须在同一模式下

{BEFORE | AFTER} <触发事件> ON <表名>:

- 触发器只能定义在基本表上，不能定义在视图上
- 当基本表的数据发生变化时，将激活定义在该表上相应触发事件的触发器
- 触发事件可以是INSERT\DELETE\UPDATE，也可以是组合
- 触发事件也可以是UPDATE OF <列名>[,<列名>]..., 即进一步指明修改哪些列时激活触发器
- AFTER\BEFORE是触发的时机

7.4 定义触发器3

1. 触发器类型：

- 行级触发器 (FOR EACH ROW)
- 语句级触发器 (FOR EACH STATEMENT)

2. 触发条件：

- 触发器被激活时，只有当<触发条件>为真时才执行<触发动作体>; 否则<触发动作体>不执行
- 如果省略“WHEN <触发条件>”，则<触发动作体>在触发器激活后立即执行

3. 触发动作体：

- 触发动作体可以是一个匿名PL/SQL过程块，也可以是对已创建存储过程的调用
- 如果是行级触发器，用户可以在过程体中使用NEW和OLD引用事件之后的新值和事件之前的旧值
- 如果是语句级触发器则不能在触发动作体中使用NEW或OLD进行引用

- 如果触发动作体执行失败，激活触发器的事件就会终止执行，触发器的目标表或触发器可能影响的其他对象不发生变化

7.5 触发器类型

行级触发器和语句级触发器：

- 例如，先在例5.11的TEACHER表上创建一个AFTER UPDATE触发器，触发事件是UPDATE语句。然后执行下述UPDATE操作：

UPDATE TEACHER SET Deptno=5;

- 假设在表TEACHER有1000行，在执行完上述的UPDATE语句后
 - 如果是语句级触发器，则触发动作只被执行一次
 - 如果是行级触发器，则触发动作将被执行1000次

触发器示例：

- [例5.21]当对表SC的Grade列进行修改时，若分数增加了10%则将此次操作记录到表SC_U(Sno,Cno,Oldgrade,Newgrade)中去。其中：Oldgrade是修改前的分数，Newgrade是修改后的分数。

```
CREATE TRIGGER SC_T
AFTER UPDATE OF Grade ON SC
REFERENCING
    OLD row AS OldTuple,
    NEW row AS NewTuple
FOR EACH ROW
WHEN (NewTuple.Grade >= 1.1*OldTuple.Grade)
INSERT INTO SC_U(Sno,Cno,OldGrade,NewGrade)
VALUES(OldTuple.Sno, OldTuple.Cno, OldTuple.Grade, NewTuple.Grade);
```

- [例5.23] 定义一个BEFORE行级触发器，为教师表Teacher定义完整性规则“教授的工资不得低于4000元，如果低于4000元，自动改为4000元”。

```
CREATE TRIGGER Insert_Or_Update_Sal
BEFORE INSERT OR UPDATE ON Teacher
/*触发事件是插入或更新操作*/
REFERENCING NEW ROW AS newTuple
FOR EACH ROW /*行级触发器*/
BEGIN /*定义触发动作体，是一个PL/SQL过程块*/
    IF (newTuple.Job = '教授') AND (newTuple.Sal < 4000)
    THEN newTuple.Sal := 4000;
    END IF;
END;
```

触发器是一种触发事件，并不是一定如断言那样采取拒绝操作

7.6 激活触发器

触发器的执行，是由触发事件激活的，并由数据库服务器自动执行。

触发器的执行顺序：

- 执行该表上的BEFORE触发器
- 激活触发器的SQL语句
- 执行该表上的AFTER触发器

7.7 删除触发器

语句：

```
1 DROP TRIGGER <触发器名> ON <表名>
```

触发器必须是一个已经创建的触发器，并且只能由具有相应权限的用户删除

7.8 断言VS触发器

	断言	触发器
1	要求完整性约束条件永远为真。	触发条件：可能有，也可能没有；可能成立，也可能不成立。
2	当数据库中的数据发生变化时，DBMS将在全数据库范围内启动断言检查。	仅当触发事件发生且触发条件成立时，DBMS才启动与之对应的触发器的执行
3	断言仅仅用于完整性约束检查，并在违反约束的情况下拒绝执行用户的数据更新操作。	触发器有助于维护数据库表中的完整性约束，尤其是在未定义主码和外码约束时。
4	断言无法对表中所做更改进行任何跟踪。	触发器可以通过执行SQL程序块或调用存储过程来跟踪表中发生的所有更改，包括用户业务逻辑的实现。
5	断言执行检查的代价较高，现代数据库系统一般不使用断言。（慎用）	触发器不仅可以用来实现复杂的完整性约束检查，还可以用于数据库安全性检查和用户业务逻辑的实现，在现代数据库系统中得到了很好的应用。